

Secure Infrastructure as Code Architecture

THE ARZENS — Advanced Track, Week 07 — Task 4

Prepared by Saqlain — AI, Automation & Security Engineering Track

1. Introduction

Infrastructure as Code (IaC) replaces manual, click-ops cloud configuration with declarative, version-controlled definitions — but that same automation can just as easily codify a security mistake and replicate it across every environment it touches. A secure IaC architecture therefore treats the Terraform/Ansible toolchain itself as part of the attack surface: state files can leak secrets, an unreviewed module can open a security group to the world, and configuration drift can silently undo a hardening control applied months earlier. This document describes the architecture built for THE ARZENS Task 5/6 deliverables: a modular Terraform configuration provisioning a hardened AWS environment, configured post-provision by Ansible.

2. Architecture Overview

The pipeline is organized in five layers: version-controlled source, a CI validation gate, the Terraform module tree, an encrypted remote state/secrets layer, and the provisioned infrastructure handed off to Ansible for configuration management. Figure 1 shows the full structure, mirroring the actual **modules/** directory (vpc, security, compute, storage) used in the Task 5 submission, so this design document and the code it describes stay in lock-step.

Secure IaC Architecture — Terraform + Ansible

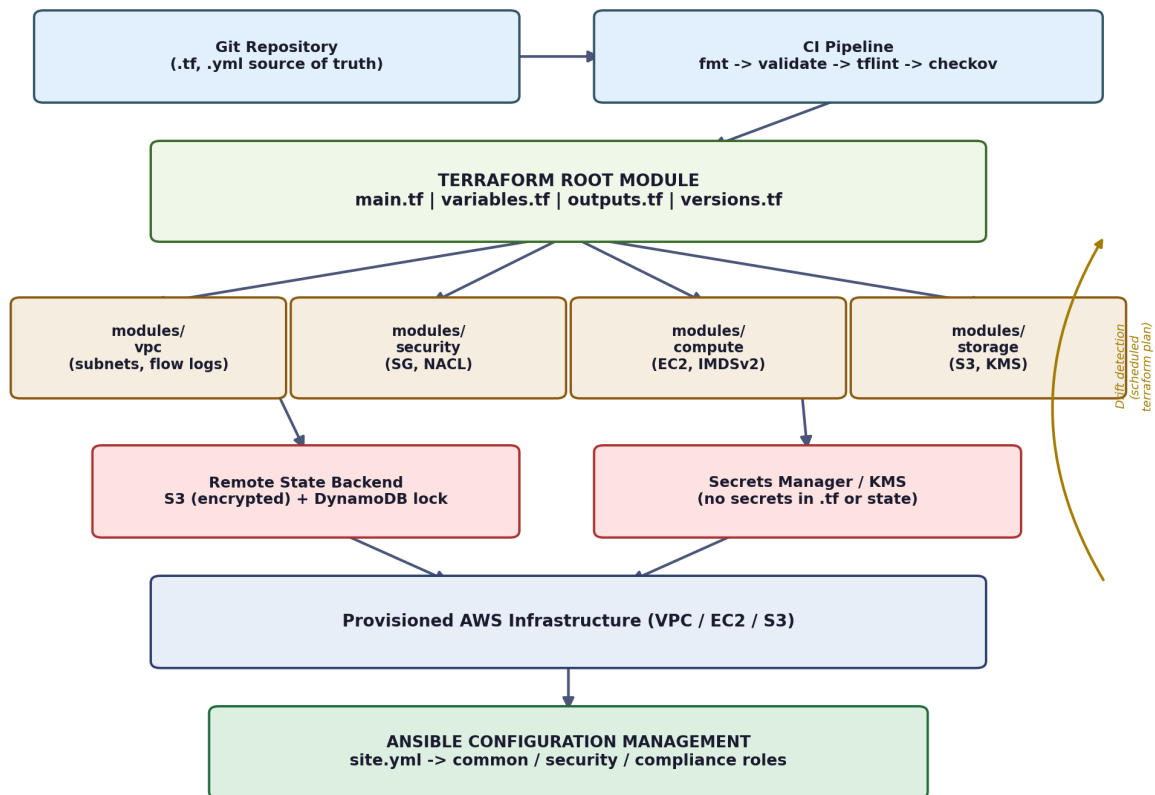


Figure 1 — Terraform module structure and Ansible hand-off.

3. Tooling: Terraform + Ansible

Terraform owns provisioning — creating the VPC, security groups, EC2 instance, and S3 bucket declaratively, so the desired end-state is always readable from source rather than reconstructed from a runbook. **Ansible** owns configuration management after the instance exists — SSH hardening, firewall rules, fail2ban, and audit logging — because these are host-internal, imperative changes better expressed as idempotent tasks than as cloud-provider resources. Splitting responsibilities this way keeps each tool doing what it is best at and keeps either layer independently re-runnable.

4. Security Controls

4.1 State file encryption

Terraform state can contain resource attributes in plaintext, including, in some providers, sensitive outputs. The architecture specifies an S3 backend with **encrypt = true** and a customer-managed KMS key, paired with a DynamoDB table for state locking so concurrent applies cannot corrupt state. In this submission the backend block is present but commented out so grading works without a pre-existing state bucket; uncommenting it is the only change needed for a real encrypted-remote-state deployment.

4.2 Secret management

No credentials are hardcoded in .tf files. **terraform.tfvars** (which holds the admin CIDR, key-pair name, and bucket suffix) is git-ignored, and a **terraform.tfvars.example** template is committed instead. In a production deployment, short-lived credentials would be sourced from AWS Secrets Manager or Vault rather than any file on disk, and the EC2 instance authenticates purely via key-pair — no passwords, no embedded API keys.

4.3 Policy enforcement

A **validate.sh** script chains `terraform fmt -check`, `terraform validate`, and, where available, **tflint** and **checkov** for static security-policy scanning (public S3 buckets, overly permissive security groups, missing encryption). Wiring this into a CI pipeline as a pre-merge gate — as shown in Figure 1 — stops an insecure change from ever reaching main, rather than catching it after deployment.

5. Drift Detection

Infrastructure drifts whenever a change is made outside Terraform — a manually-added security-group rule, a console-edited S3 policy. The architecture addresses this with a scheduled (nightly) `terraform plan` run against the live environment; any non-empty diff is drift and is surfaced to the team before it becomes a security gap. This mirrors the same principle used in Task 3's IOC health-check automation: continuously re-verify assumed state rather than trusting it stayed correct.

6. Rollback Procedures

Because S3 bucket versioning is enabled on both the application bucket (Task 5) and, once configured, the state bucket itself, a bad apply can be rolled back in two ways: (1) restore the previous state object version and re-run `terraform apply` against it, or (2) check out the prior Git commit's .tf files and re-apply — since the module tree is the single source of truth, rolling back code and rolling back infrastructure are the same operation.

7. Compliance Validation

Compliance is validated at two points: pre-deployment via the **checkov/tflint** policy scan described above, and post-deployment via the Ansible **compliance** role, which runs CIS Distribution Independent Linux Benchmark-aligned checks (root SSH login disabled, firewall active, audit rules loaded) against the live instance and renders a pass/fail report. Running both closes the loop between "the code is secure" and "the running system is actually configured the way the code says it should be."

8. Reference Architecture

This design follows **HashiCorp's Security Best Practices** guidance for Terraform: remote encrypted state with locking, least-privilege IAM for any automation identity, no secrets committed to version control, and small, composable modules over one large monolithic configuration — the same module boundaries (vpc / security / compute / storage) recommended for teams beyond a single-developer scope.

9. Conclusion

Treating the IaC toolchain itself as security-relevant infrastructure — encrypting state, externalizing secrets, gating changes through policy scanning, and continuously checking for drift — turns Terraform and Ansible from a convenience into a security control. The corresponding implementation is submitted alongside this document as Tasks 5 and 6.